algorithm template v3.0

目录

1. 数学与数论基础
2. 搜索与图上搜索
3. 动态规划 DP
4. 图论
5. 高级数据结构与常用技巧
6. 字符串算法
7. 树上算法

0. 通用头与习惯

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int,int>;
const int INF = 0x3f3f3f3f;
const ll LINF = (ll)4e18;

int main(){
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  return 0;
}
```

1. 数学与数论基础

1.1 快速幂 / 逆元（费马）

```
ll qpow(ll a,ll b,ll mod){
  ll r=1%mod;
  a%=mod;
  while(b){
    if(b&1) r=r*a%mod; // 当前位为 1，乘到答案里
    a=a*a%mod;          // 底数平方
    b>>=1;              // 处理下一位
  }
  return r;
}
// mod 为质数时 inv(a)=a^(mod-2)
ll inv_mod(ll a,ll mod){ return qpow(a,mod-2,mod); }
```

1.2 gcd / exgcd / 线性同余

```
ll gcdll(ll a,ll b){ return b?gcdll(b,a%b):a; }

ll exgcd(ll a,ll b,ll &x,ll &y){
  if(!b){ x=1; y=0; return a; }
  ll x1,y1,g=exgcd(b,a%b,x1,y1);
  x=y1;
  y=x1-a/b*y1;
  return g;
}

// 解 ax ≡ c (mod m)，返回是否有解；有解时给出最小非负解 x0（模 m/g）
bool linear_congruence(ll a,ll c,ll m,ll &x0,ll &mod2){
  ll x,y,g=exgcd(a,m,x,y);
  if(c%g) return false;
  mod2=m/g; // 解空间模数
  x0=(__int128)x*(c/g)%mod2;
  if(x0<0) x0+=mod2;
  return true;
}
```

1.3 素数筛（埃氏 / 线性筛）

```
// 埃氏筛 O(n log log n)
vector<int> eratosthenes(int n){
  vector<int> is(n+1,1), p; is[0]=is[1]=0;
  for(int i=2;i<=n;i++) if(is[i]){
    p.push_back(i);
    if((ll)i*i<=n) for(int j=i*i;j<=n;j+=i) is[j]=0;
  }
  return p;
}

// 线性筛 O(n)
vector<int> linear_sieve(int n){
  vector<int> lp(n+1), p;
  for(int i=2;i<=n;i++){
    if(!lp[i]){
      lp[i]=i;
      p.push_back(i);
    }
    for(int x:p){
      if(x>lp[i] || (ll)i*x>n) break;
      lp[i*x]=x;
    }
  }
  return p;
}
```

1.4 组合数（阶乘 + 逆元）

```
const int MOD = 1e9+7;
vector<ll> fac,ifac;
void init_comb(int n){
  fac.assign(n+1,1); ifac.assign(n+1,1);
  for(int i=1;i<=n;i++) fac[i]=fac[i-1]*i%MOD;
  ifac[n]=qpow(fac[n],MOD-2,MOD);
  for(int i=n;i;i--) ifac[i-1]=ifac[i]*i%MOD;
}
ll C(int n,int k){
  if(k<0||k>n) return 0;
  return fac[n]*ifac[k]%MOD*ifac[n-k]%MOD;
}
```

2. 搜索与图上搜索

2.1 DFS（连通块/回溯）

```
int n,m; vector<vector<int>> g; vector<int> vis;
void dfs(int u){
  vis[u]=1;
  for(int v:g[u]) if(!vis[v]) dfs(v);
}
```

2.2 BFS（无权最短路）

```
vector<int> bfs(int s,const vector<vector<int>>& g){
  int n=g.size()-1; vector<int> d(n+1,-1);
  queue<int> q; d[s]=0; q.push(s);
  while(!q.empty()){
    int u=q.front(); q.pop();
    for(int v:g[u]) if(d[v]==-1) d[v]=d[u]+1,q.push(v);
  }
  return d;
}
```

2.3 网格 BFS（4方向）

```
int dx[4]={-1,0,1,0}, dy[4]={0,1,0,-1};
int grid_bfs(vector<string>& a,int sx,int sy,int tx,int ty){
  int n=a.size(), m=a[0].size();
  vector<vector<int>> d(n,vector<int>(m,-1));
  queue<pii> q; d[sx][sy]=0; q.push({sx,sy});
  while(!q.empty()){
    auto [x,y]=q.front(); q.pop();
    if(x==tx&&y==ty) return d[x][y];
    for(int k=0;k<4;k++){
      int nx=x+dx[k], ny=y+dy[k];
      if(nx<0||nx>=n||ny<0||ny>=m||a[nx][ny]=='#'||d[nx][ny]!=-1) continue;
      d[nx][ny]=d[x][y]+1; q.push({nx,ny});
    }
  }
  return -1;
}
```

2.4 回溯剪枝（排列示意）

```
int n; vector<int> path,used;
void bt(){
  if((int)path.size()==n){ /* 记录答案 */ return; }
  for(int x=1;x<=n;x++){
    if(used[x]) continue;
    // 可行性剪枝：if(!ok(x)) continue;
    used[x]=1; path.push_back(x);
    bt();
    path.pop_back(); used[x]=0;
  }
}
```

2.5 A*（状态图通用写法）

思路：维护 `f = g + h`， `h` 需尽量接近真实代价且不超过真实代价（可采纳启发式）。

```
struct Node{
  int state, g, f;
  bool operator<(const Node& o) const{ return f>o.f; } // 小根堆
};

int h(int s){ return 0; } // 按题目设计启发函数

int astar(int S,int T,const vector<vector<pair<int,int>>>& gph){
  int n=gph.size()-1;
  vector<int> best(n+1,INF);
  priority_queue<Node> pq;
  best[S]=0; pq.push({S,0,h(S)});
  while(!pq.empty()){
    auto [u,gu,fu]=pq.top(); pq.pop();
    if(u==T) return gu;
    if(gu!=best[u]) continue;
    for(auto [v,w]:gph[u]){
      int ng=gu+w;
      if(ng<best[v]) best[v]=ng,pq.push({v,ng,ng+h(v)});
    }
  }
  return -1;
}
```

3. 动态规划 DP

3.1 线性 DP（最长上升子序列 LIS, O(nlogn)）

```
int LIS(const vector<int>& a){
  vector<int> d;
  for(int x:a){
    auto it=lower_bound(d.begin(),d.end(),x);
    if(it==d.end()) d.push_back(x); else *it=x;
  }
  return (int)d.size();
}
```

3.2 0/1 背包

```
// w[i], val[i], 容量 V
vector<int> dp(V+1,0);
for(int i=1;i<=n;i++)
  for(int v=V;v>=w[i];v--)
    dp[v]=max(dp[v],dp[v-w[i]]+val[i]);
```

3.3 完全背包

```
vector<int> dp(V+1,0);
// 先物品后容量，内层正序枚举容量保证每件物品可选多次
for(int i=1;i<=n;i++)
  for(int v=w[i];v<=V;v++)
    dp[v]=max(dp[v],dp[v-w[i]]+val[i]);
```

3.4 多重背包（二进制拆分）

```
vector<int> dp(V+1,0);
// 先物品后容量，内层逆序枚举容量保证每件物品只能选 k 件
for(int i=1;i<=n;i++){
  int c=cnt[i],k=1;
  while(k<=c){
    int ww=k*w[i], vv=k*val[i];
    for(int v=V;v>=ww;v--) dp[v]=max(dp[v],dp[v-ww]+vv);
    c-=k; k<<=1;
  }
  if(c){
    int ww=c*w[i], vv=c*val[i];
    for(int v=V;v>=ww;v--) dp[v]=max(dp[v],dp[v-ww]+vv);
  }
}
```

3.5 区间 DP（合并石子原型）

```
// dp[l][r]: 合并区间 [l,r] 最小代价
for(int len=2;len<=n;len++)
  for(int l=1,r;l+len-1<=n;l++){
  r=l+len-1; dp[l][r]=INF;
  for(int k=l;k<r;k++)
    dp[l][r]=min(dp[l][r],dp[l][k]+dp[k+1][r]+sum[r]-sum[l-1]);
  }
```

3.6 状压 DP（TSP 原型）

```
int N=1<<n;
vector<vector<int>> dp(N,vector<int>(n,INF));
dp[1<<s][s]=0;
for(int mask=0;mask<N;mask++) for(int u=0;u<n;u++) if(mask>>u&1){
  if(dp[mask][u]==INF) continue;
  for(int v=0;v<n;v++) if(!(mask>>v&1))
    dp[mask|1<<v][v]=min(dp[mask|1<<v][v],dp[mask][u]+w[u][v]);
}
```

3.7 树形 DP（树上选点通用）

```
int n; vector<vector<int>> tr;
vector<array<int,2>> f; // f[u][0/1]
void dfs(int u,int fa){
  f[u][1]=1; // 选 u
  for(int v:tr[u]) if(v!=fa){
    dfs(v,u);
    f[u][0]+=max(f[v][0],f[v][1]);
    f[u][1]+=f[v][0];
  }
}
```

4. 图论

图论模板默认以“邻接表 + 1-index”组织

4.1 并查集 DSU

```
struct DSU{
  vector<int> p,sz;
  DSU(int n=0){ init(n); }
  void init(int n){
    p.resize(n+1);
    sz.assign(n+1,1);
    iota(p.begin(),p.end(),0);
  }
  int find(int x){ return p[x]==x?x:p[x]=find(p[x]); }
  bool unite(int a,int b){
    a=find(a);
    b=find(b);
    if(a==b) return false;
    if(sz[a]<sz[b]) swap(a,b); // 按秩合并
    p[b]=a;
    sz[a]+=sz[b];
    return true;
  }
};
```

4.2 最小生成树 Kruskal / Prim

```
// Kruskal，适用于稀疏图，边列表存储
struct E{int u,v,w; bool operator<(const E& o)const{return w<o.w;}};
ll kruskal(int n, vector<E>& e){
  sort(e.begin(),e.end());
  DSU d(n);
  ll ans=0;
  int cnt=0;
  for(auto &x:e){
    if(!d.unite(x.u,x.v)) continue;
    ans+=x.w;
    if(++cnt==n-1) break;
  }
  return (cnt==n-1?ans:-1);
}

// Prim（稠密/邻接表均可）
ll prim(int n, vector<vector<pair<int,int>>>& g){
  vector<int> vis(n+1,0), dis(n+1,INF);
  dis[1]=0;
  priority_queue<pii,vector<pii>,greater<pii>> pq; pq.push({0,1});
  ll ans=0; int cnt=0;
  while(!pq.empty()){
    auto [d,u]=pq.top();
    pq.pop();
    if(vis[u]) continue;
    vis[u]=1;
    ans+=d;
    cnt++;
    for(auto [v,w]:g[u]){
      if(vis[v] || w>=dis[v]) continue;
      dis[v]=w;
      pq.push({w,v});
    }
  }
  return (cnt==n?ans:-1);
}
```

4.3 最短路：Dijkstra / Bellman-Ford / SPFA / Floyd

```
// Dijkstra: 要求非负边，每次贪心确定当前未确定点中距离最小者
vector<ll> dijkstra(int s,const vector<vector<pair<int,int>>>& g){
  int n=g.size()-1;
  vector<ll> dis(n+1,LINF);
  vector<int> vis(n+1,0);
  priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>> pq;
  dis[s]=0;
  pq.push({0,s});
  while(!pq.empty()){
    auto [d,u]=pq.top();
    pq.pop();
    if(vis[u]) continue;
    vis[u]=1;
    for(auto [v,w]:g[u]){
      if(dis[v]<=d+w) continue;
      dis[v]=d+w;
      pq.push({dis[v],v});
    }
  }
  return dis;
}

// Bellman-Ford: 可判负环（从 s 可达）
struct Edge{int u,v,w;};
bool bellman_ford(int n,int s,const vector<Edge>& edges,vector<ll>& dis){
  dis.assign(n+1,LINF); dis[s]=0;
  for(int i=1;i<=n-1;i++){
    bool any=0;
    for(auto &e:edges) if(dis[e.u]<LINF && dis[e.v]>dis[e.u]+e.w)
      dis[e.v]=dis[e.u]+e.w, any=1;
    if(!any) break;
  }
  for(auto &e:edges) if(dis[e.u]<LINF && dis[e.v]>dis[e.u]+e.w) return false;
  return true;
}

// SPFA: 在BF上优化，使用队列维护“待松弛点”，可判负环
bool spfa_neg_cycle(int n,int s,const vector<vector<pair<int,int>>>& g,vector<ll>& dis){
  vector<int> inq(n+1,0), cnt(n+1,0);
  dis.assign(n+1,LINF);
  queue<int> q;
```

```
  dis[s]=0;
  q.push(s);
  inq[s]=1;
  cnt[s]=1;
  while(!q.empty()){
    int u=q.front();
    q.pop();
    inq[u]=0;
    for(auto [v,w]:g[u]) if(dis[u]<LINF && dis[v]>dis[u]+w){
      dis[v]=dis[u]+w;
      if(!inq[v]){
        q.push(v);
        inq[v]=1;
        if(++cnt[v]>n) return false; // 超过 n 次说明存在可达负环
      }
    }
  }
  return true;
}

// Floyd: 全源最短路 O(n^3)
void floyd(int n, vector<vector<ll>>& d){
  for(int k=1;k<=n;k++)
    for(int i=1;i<=n;i++) if(d[i][k]<LINF)
      for(int j=1;j<=n;j++) if(d[k][j]<LINF)
        d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
}
```

4.4 Johnson（可带负边的全源最短路）

要点：

1. 加超级源 `0 -> i (0)` ，跑一次 SPFA/BF 得势函数 `h[i]` 。
2. 重标定 `w'(u,v)=w(u,v)+h[u]-h[v]` ，保证非负。
3. 每个源点跑 Dijkstra，再还原原图距离。

```
// 只给核心：假设 h[] 已求出且无负环
for(int u=1;u<=n;u++) for(auto &[v,w]:g[u]) w = w + h[u] - h[v]; // 重标定

for(int s=1;s<=n;s++){
  auto d = dijkstra(s,g); // 在重标定图上
  for(int t=1;t<=n;t++) if(d[t]<LINF)
    d[t] = d[t] - h[s] + h[t]; // 还原
}
```

4.5 拓扑排序 + DAG DP

```
vector<int> topo(int n, vector<vector<int>>& g){
  vector<int> indeg(n+1), ord; queue<int> q;
  for(int u=1;u<=n;u++) for(int v:g[u]) indeg[v]++;
  for(int i=1;i<=n;i++) if(!indeg[i]) q.push(i);
  while(!q.empty()){
    int u=q.front(); q.pop(); ord.push_back(u);
    for(int v:g[u]) if(--indeg[v]==0) q.push(v);
  }
  return ord; // size<n 则有环
}

// DAG 上最长路（边权可正负，需无环）
vector<ll> dag_longest(int n,int s, vector<vector<pair<int,int>>>& g){
  vector<vector<int>> g0(n+1);
  for(int u=1;u<=n;u++) for(auto [v,w]:g[u]) g0[u].push_back(v);
  auto ord = topo(n,g0);
  vector<ll> dp(n+1,-LINF); dp[s]=0;
  for(int u:ord) if(dp[u]>-LINF)
    for(auto [v,w]:g[u]) dp[v]=max(dp[v],dp[u]+w);
  return dp;
}
```

4.6 Tarjan 强连通分量（完整背景变量）

```
const int N = 200005;
int n,m;
vector<int> g[N];

int dfn[N], low[N], ts;
int scc_id[N], scc_cnt, sz[N];
bool in_stk[N];
stack<int> stk;

void tarjan(int u){
  dfn[u]=low[u]=++ts;
  stk.push(u); in_stk[u]=1;
  for(int v:g[u]){
    if(!dfn[v]){
      tarjan(v);
      low[u]=min(low[u],low[v]);
    }else if(in_stk[v]){
      low[u]=min(low[u],dfn[v]);
    }
  }
  if(dfn[u]==low[u]){
    ++scc_cnt;
    while(true){
      int x=stk.top(); stk.pop(); in_stk[x]=0;
      scc_id[x]=scc_cnt; sz[scc_cnt]++;
      if(x==u) break;
    }
  }
}

void build_scc(){
  for(int i=1;i<=n;i++) if(!dfn[i]) tarjan(i);
}
```

4.7 SCC 缩点 + 拓扑 DP（常见优化套路）

```
vector<vector<int>> dag; vector<int> indeg;
void build_dag(){
  dag.assign(scc_cnt+1,{}); indeg.assign(scc_cnt+1,0);
  for(int u=1;u<=n;u++) for(int v:g[u]){
    int a=scc_id[u], b=scc_id[v];
    if(a!=b) dag[a].push_back(b);
  }
  for(int i=1;i<=scc_cnt;i++){
    sort(dag[i].begin(),dag[i].end());
    dag[i].erase(unique(dag[i].begin(),dag[i].end()),dag[i].end());
    for(int v:dag[i]) indeg[v]++;
  }
}

// 例：点权为 val_scc[i]，求 DAG 最长路
vector<ll> topo_dp(const vector<ll>& val_scc){
  vector<ll> f(scc_cnt+1,-LINF); queue<int> q;
  for(int i=1;i<=scc_cnt;i++) if(!indeg[i]) q.push(i),f[i]=val_scc[i];
  while(!q.empty()){
    int u=q.front(); q.pop();
    for(int v:dag[u]){
      f[v]=max(f[v],f[u]+val_scc[v]);
      if(--indeg[v]==0) q.push(v);
    }
  }
  return f;
}
```

4.8 差分约束（完整建图 + SPFA 判环）

约束转化：

- `x_v - x_u <= w` 转 `u -> v (w)`
- `x_v - x_u >= w` 转 `v -> u (-w)`
- `x_v - x_u = w` 转 `u -> v (w)` 和 `v -> u (-w)`
  如果是求最大路径，则建负边 `x_v - x_u <= w` 转 `v -> u (-w)`，其他同理。
  做法：加超级源 `0 -> i (0)`，跑最短路；若出现负环则无解。

```cpp
int n,m;
struct DEdge{ int v,w; };           // u->v, xv-xu<=w
vector<vector<DEdge>> g;
vector<int> dist, inq, cnt;
const int INF2 = 1e9;

bool spfa_diff(int s){
  dist.assign(n+1,INF2);
  inq.assign(n+1,0);
  cnt.assign(n+1,0);
  queue<int> q;
  dist[s]=0; q.push(s); inq[s]=1; cnt[s]=1;
  while(!q.empty()){
    int u=q.front(); q.pop(); inq[u]=0;
    for(auto &e:g[u]){
      if(dist[u]!=INF2 && dist[e.v]>dist[u]+e.w){
        dist[e.v]=dist[u]+e.w;
        if(!inq[e.v]){
          q.push(e.v); inq[e.v]=1;
          if(++cnt[e.v]>n) return false; // 负环
        }
      }
    }
  }
  return true;
}

int main(){
  cin>>n>>m;
  g.assign(n+1,{});
  for(int i=0;i<m;i++){
    int u,v,w; cin>>u>>v>>w; // xv-xu<=w
    g[u].push_back({v,w});
  }
  int S=0; g.resize(n+1);
  for(int i=1;i<=n;i++) g[S].push_back({i,0});
  if(!spfa_diff(S)) cout<<"NO\n";
  else for(int i=1;i<=n;i++) cout<<dist[i]<<" \n"[i==n];
}
```

4.9 最大流 Dinic（层次图 BFS + 阻塞流 DFS）

```
struct Dinic{
  struct E{int to,cap,rev;};
  int n,s,t; vector<vector<E>> g; vector<int> lv,it;
  Dinic(int n):n(n),g(n+1),lv(n+1),it(n+1){}
  void addEdge(int u,int v,int c){
    E a{v,c,(int)g[v].size()}, b{u,0,(int)g[u].size()};
    g[u].push_back(a); g[v].push_back(b);
  }
  bool bfs(){
    fill(lv.begin(),lv.end(),-1);
    queue<int> q; lv[s]=0; q.push(s);
    while(!q.empty()){
      int u=q.front(); q.pop();
      for(auto &e:g[u]) if(e.cap>0 && lv[e.to]==-1)
        lv[e.to]=lv[u]+1,q.push(e.to);
    }
    return lv[t]!=-1;
  }
  int dfs(int u,int f){
    if(u==t) return f;
    for(int &i=it[u]; i<(int)g[u].size(); i++){
      E &e=g[u][i];
      if(e.cap<=0 || lv[e.to]!=lv[u]+1) continue;
      int ret=dfs(e.to,min(f,e.cap));
      if(ret){ e.cap-=ret; g[e.to][e.rev].cap+=ret; return ret; }
    }
    return 0;
  }
  ll maxflow(int S,int T){
    s=S; t=T; ll flow=0;
    while(bfs()){
      fill(it.begin(),it.end(),0);
      while(int f=dfs(s,INF)) flow+=f;
    }
    return flow;
  }
};
```

5. 高级数据结构与常用技巧

5.1 前缀和 / 差分（1D + 2D）

```
// 1D 前缀和 [l,r]=pre[r]-pre[l-1]
for(int i=1;i<=n;i++) pre[i]=pre[i-1]+a[i];

// 1D 差分: 区间 [l,r] += x
diff[l]+=x; diff[r+1]-=x;

// 2D 前缀和
pre[i][j]=pre[i-1][j]+pre[i][j-1]-pre[i-1][j-1]+a[i][j];

// 2D 差分: 子矩阵 (x1,y1)-(x2,y2) += v
d[x1][y1]+=v; d[x1][y2+1]-=v; d[x2+1][y1]-=v; d[x2+1][y2+1]+=v;
```

5.2 双指针 / 滑动窗口

```
// 最长满足条件子数组
int ans=0;
for(int l=1,r=1;r<=n;r++){
  // 加入 a[r]
  while(/* 不合法 */){
    // 删除 a[l]
    l++;
  }
  ans=max(ans,r-l+1);
}
```

5.3 单调栈（右侧第一个更大）

```
vector<int> nxt(n+1,0); stack<int> st;
for(int i=1;i<=n;i++){
  while(!st.empty() && a[i]>a[st.top()]) nxt[st.top()]=i,st.pop();
  st.push(i);
}
```

5.4 单调队列（固定窗口最值）

```
deque<int> q;
for(int i=1;i<=n;i++){
  while(!q.empty() && q.front()<=i-k) q.pop_front();
  while(!q.empty() && a[q.back()]<=a[i]) q.pop_back();
  q.push_back(i);
  if(i>=k) ans.push_back(a[q.front()]);
}
```

5.5 树状数组 Fenwick

Fenwick 适合“前缀可加性”问题，单次修改/查询复杂度 O(logn)，代码短、常数小。

```
struct BIT{
  int n; vector<ll> t;
  BIT(int n=0){ init(n); }
  void init(int n_){
    n=n_;
    t.assign(n+1,0);
  }
  int lowbit(int x){ return x&-x; }
  void add(int i,ll v){
    for(;i<=n;i+=lowbit(i)) t[i]+=v; // 往上更新所有受影响节点
  }
  ll sum(int i){
    ll r=0;
    for(;i;i-=lowbit(i)) r+=t[i];
    return r;
  }
  ll rangeSum(int l,int r){ return sum(r)-sum(l-1); }
};

// 区间加、单点查：维护差分
// add(l,v), add(r+1,-v), query(x)=sum(x)
```

5.6 线段树（区间加 + 区间和）

简述：区间修改 + 区间查询的经典结构；懒标记表示“这段还没下传的增量”。

```
struct SegTree{
  struct Node{ ll sum,lz; };
  int n; vector<Node> tr;
  SegTree(int n=0){ init(n); }
  void init(int n_){
    n=n_;
    tr.assign(4*n+4,{0,0});
  }
  void push(int p,int l,int r){
    if(!tr[p].lz||l==r) return;
    int m=(l+r)>>1;
    int lc=p<<1, rc=lc|1;
    ll z=tr[p].lz;
    // 下传懒标记，左右儿子区间和都要同步增加
    tr[lc].lz+=z; tr[rc].lz+=z;
    tr[lc].sum+=z*(m-l+1); tr[rc].sum+=z*(r-m);
    tr[p].lz=0;
  }
  void add(int p,int l,int r,int ql,int qr,ll v){
    if(ql<=l&&r<=qr){
      tr[p].sum+=v*(r-l+1);
      tr[p].lz+=v;
      return;
    }
    push(p,l,r);
    int m=(l+r)>>1;
    if(ql<=m) add(p<<1,l,m,ql,qr,v);
    if(qr>m) add(p<<1|1,m+1,r,ql,qr,v);
    tr[p].sum=tr[p<<1].sum+tr[p<<1|1].sum;
  }
  ll query(int p,int l,int r,int ql,int qr){
    if(ql<=l&&r<=qr) return tr[p].sum;
    push(p,l,r);
    int m=(l+r)>>1;
    ll ans=0;
    if(ql<=m) ans+=query(p<<1,l,m,ql,qr);
    if(qr>m) ans+=query(p<<1|1,m+1,r,ql,qr);
    return ans;
  }
};
```

5.7 排序模板

```
sort(a.begin(),a.end());
sort(a.begin(),a.end(),greater<int>()); // 降序
sort(v.begin(),v.end(),
[](const Node& A,const Node& B){ return A.w<B.w; }); // 按 Node.w 升序
```

5.8 高精度（核心版：加/乘）

```
// 非负整数，低位在前
vector<int> addBig(const string& s1,const string& s2){
  vector<int> A,B,C; int t=0;
  for(int i=(int)s1.size()-1;i>=0;i--) A.push_back(s1[i]-'0');
  for(int i=(int)s2.size()-1;i>=0;i--) B.push_back(s2[i]-'0');
  for(int i=0;i<(int)A.size()||i<(int)B.size()||t;i++){
    if(i<(int)A.size()) t+=A[i]; if(i<(int)B.size()) t+=B[i];
    C.push_back(t%10); t/=10;
  }
  return C;
}

vector<int> mulBig(const string& s1,const string& s2){
  vector<int> A,B,C;
  for(int i=(int)s1.size()-1;i>=0;i--) A.push_back(s1[i]-'0');
  for(int i=(int)s2.size()-1;i>=0;i--) B.push_back(s2[i]-'0');
  C.assign(A.size()+B.size()+1,0);
  for(int i=0;i<(int)A.size();i++) for(int j=0;j<(int)B.size();j++) C[i+j]+=A[i]*B[j];
  for(int i=0;i+1<(int)C.size();i++) C[i+1]+=C[i]/10, C[i]%=10;
  while(C.size()>1&&C.back()==0) C.pop_back();
  return C;
}
```

5.9 ST 表

```
// 预处理 O(nlogn)，查询 O(1)，用于区间最大，最小，gcd
const int N = 100005;
int n, m;
int a[N];
int st[N][20];
int lg[N];

int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i];
    // 预处理 log
    lg[1] = 0;
    for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
    // 初始化长度为 1 的区间
    for (int i = 1; i <= n; i++)  st[i][0] = a[i];
    // 预处理 ST 表
    for (int j = 1; j <= lg[n]; j++) {
        for (int i = 1; i + (1 << j) - 1 <= n; i++) {
            st[i][j] = min(st[i][j - 1],
                           st[i + (1 << (j - 1))][j - 1]);
        }
    }
    while (m--) {// 查询
        int l, r; cin >> l >> r;
        int k = lg[r - l + 1];
        int ans = min(st[l][k],st[r - (1 << k) + 1][k]);
        cout << ans << '\n';
    }
}
```

6. 字符串算法

6.1 KMP

```
vector<int> get_nxt(const string& p){
  int m=p.size(); vector<int> nxt(m,0);
  for(int i=1,j=0;i<m;i++){
    while(j&&p[i]!=p[j]) j=nxt[j-1];
    if(p[i]==p[j]) j++;
    nxt[i]=j;
  }
  return nxt;
}

vector<int> kmp_match(const string& s,const string& p){
  vector<int> nxt=get_nxt(p), pos;
  for(int i=0,j=0;i<(int)s.size();i++){
    while(j&&s[i]!=p[j]) j=nxt[j-1];
    if(s[i]==p[j]) j++;
    if(j==(int)p.size()) pos.push_back(i-j+2), j=nxt[j-1]; // 1-index
  }
  return pos;
}
```

6.2 Trie（小写字母）

```
struct Trie{
  static const int SIG=26;
  struct Node{ int ch[SIG]; int cnt; Node(){ memset(ch,0,sizeof ch); cnt=0; } };
  vector<Node> t;
  Trie(){ t.push_back(Node()); t.push_back(Node()); } // 1 为根
  void insert(const string& s){
    int u=1;
    for(char c:s){
      int x=c-'a';
      if(!t[u].ch[x]) t[u].ch[x]=t.size(), t.push_back(Node());
      u=t[u].ch[x];
    }
    t[u].cnt++;
  }
  int query(const string& s){
    int u=1;
    for(char c:s){
      int x=c-'a'; if(!t[u].ch[x]) return 0;
      u=t[u].ch[x];
    }
    return t[u].cnt;
  }
};
```

6.3 字符串哈希（单模）

```
const ll MODH=1000000007, BASE=131;
vector<ll> H,PW;
void build_hash(const string& s){
  int n=s.size(); H.assign(n+1,0); PW.assign(n+1,1);
  for(int i=1;i<=n;i++) H[i]=(H[i-1]*BASE+s[i-1])%MODH, PW[i]=PW[i-1]*BASE%MODH;
}
ll get_hash(int l,int r){ // 1-index
  return (H[r]-H[l-1]*PW[r-l+1]%MODH+MODH)%MODH;
}
```

7. 树上算法

7.1 LCA 倍增

```
const int MAXN=200005, LOG=20;
vector<int> tr[MAXN];
int up[MAXN][LOG], dep[MAXN];

void dfs_lca(int u,int fa){
  up[u][0]=fa; dep[u]=dep[fa]+1;
  for(int k=1;k<LOG;k++) up[u][k]=up[up[u][k-1]][k-1];
  for(int v:tr[u]) if(v!=fa) dfs_lca(v,u);
}

int lca(int a,int b){
  if(dep[a]<dep[b]) swap(a,b);
  for(int k=LOG-1;k>=0;k--) if(dep[up[a][k]]>=dep[b]) a=up[a][k];
  if(a==b) return a;
  for(int k=LOG-1;k>=0;k--) if(up[a][k]!=up[b][k]) a=up[a][k], b=up[b][k];
  return up[a][0];
}
```

7.2 树上距离（带边权）

```
vector<pair<int,int>> g2[MAXN];
ll dis_root[MAXN];
void dfs_dis(int u,int fa){
  for(auto [v,w]:g2[u]) if(v!=fa){
    dis_root[v]=dis_root[u]+w;
    dfs_dis(v,u);
  }
}
// dist(u,v)=dis_root[u]+dis_root[v]-2*dis_root[lca(u,v)]
```

8. 补充：二分与 STL 高频

```
// 二分答案：最小可行
int l=0,r=1e9;
while(l<r){
  int mid=l+(r-l)/2;
  if(check(mid)) r=mid;
  else l=mid+1;
}

// lower_bound / upper_bound
auto it1=lower_bound(a.begin(),a.end(),x); // >=x
auto it2=upper_bound(a.begin(),a.end(),x); // >x
```

9. 赛场 WSL+VSCode 环境配置

9.1 快速安装

WSL 安装（管理员 PowerShell）

```
wsl --install -d Ubuntu
# 首次启动会提示输入用户名和密码
```

GCC 安装

```
wsl
sudo apt update && sudo apt install -y build-essential gdb
g++ --version  # 验证
```

扩展安装（VSCode）

按 `Ctrl + Shift + X`，安装：`Remote - WSL`，`C/C++`，`Code Runner`

9.2 连接 VSCode

1. `Ctrl + Shift + P` → `Remote-WSL: New WSL Window`
2. 打开项目文件夹： `Ctrl + K Ctrl + O` → `/home/username/...`
3. 验证终端显示 `user@hostname:~$`

9.3 配置编译和调试

在项目根目录创建 `.vscode/tasks.json`：

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "C++: Build with g++ (WSL)",
      "type": "shell",
      "command": "g++",
      "args": [
        "-std=c++20", "-Wall", "-Wextra", "-g",
        "${file}", "-o", "${fileDirname}/${fileBasenameNoExtension}.out"
      ],
      "group": {"kind": "build", "isDefault": true},
      "problemMatcher": {
        "owner": "cpp",
        "fileLocation": ["relative", "${workspaceFolder}"],
        "pattern": {
          "regexp": "^(.*):(\\d+):(\\d+):\\s+(warning|error):\\s+(.*)$",
          "file": 1, "line": 2, "column": 3, "severity": 4, "message": 5
        }
      }
    }
  ]
}
```

创建 `.vscode/launch.json`：

```json
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug C++ (WSL)",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}/${fileBasenameNoExtension}.out",
      "stopAtEntry": false,
      "cwd": "${workspaceFolder}",
      "MIMode": "gdb",
      "miDebuggerPath": "/usr/bin/gdb",
      "preLaunchTask": "C++: Build with g++ (WSL)"
    }
  ]
}
```

9.4 快速参考

操作	快捷键
编译	`Ctrl + Shift + B`
运行	`Ctrl + Alt + N`
调试	`F5`
终端	`Ctrl + ` `
错误	`Ctrl + Shift + M`

常见问题

- G++ 找不到： `sudo apt install -y build-essential`
- WSL 连不上：重启 VSCode 或重新启用虚拟机平台（控制面板 → Windows 功能）
- Permission denied：文件移到 home 目录或 `/tmp`